

Arquitectura de Microservicios con Contenedores

Docentes

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Domain-Driven Design

Objetivos de la clase

Al finalizar esta clase vas a poder:

- Explicar por qué cortar por entidades o tablas produce malos microservicios.
- Construir un **Ubiquitous Language** y detectar conflictos de significado.
- Identificar **subdominios** (Core / Supporting / Generic) y proponer **Bounded Contexts**.
- Mapear relaciones entre contextos (ACL, OHS, Customer/Supplier, ...).
- Aplicar un mínimo táctico: Entity, Value Object, Aggregate, Command, Domain Event.
- Guiar un **EventStorming** y derivar BCs, eventos y candidatos a servicio.

The primary focus should be on **domain** and domain logic

Complex domain design should be based on a **model**

– Eric Evans, *Domain-Driven Design*

**¿Por qué fallan los
"microservicios por
entidad"?**

El anti-patrón clásico

Nos piden “pasar a microservicios”. El equipo mira el esquema y corta:

Tabla / entidad	“Servicio” resultante
pedidos	OrderService
clientes	CustomerService
productos	ProductService
pagos	PaymentService
envios	ShippingService

Cada tabla tiene su API y su DB. ¿Listo?

¿Qué sale mal?

- **Lenguaje confuso:** "Pedido" significa distinto en Ventas, Facturación y Logística, pero todos comparten el mismo modelo.
- **Acoplamiento por datos:** un cambio de columna rompe a cuatro equipos (monolito distribuido).
- **Transacciones imposibles:** "reservar stock + cobrar + despachar" ya no cabe en una sola DB.
- **Equipos desalineados:** la arquitectura no refleja cómo habla el negocio (ley de Conway).

Cortar por entidad/tabla optimiza el esquema relacional, no el dominio.

Ley de Conway + modelo compartido

- **Conway:** la estructura del sistema copia la estructura de comunicación de la organización.
- Si el negocio tiene áreas con **reglas y vocabularios distintos**, forzar un solo modelo "Pedido" global:
 - oculta los límites reales,
 - concentra el acoplamiento,
 - vuelve frágil cualquier "desacople" por red.

Necesitamos una forma de descubrir límites desde el negocio, no desde las tablas.

→ Domain-Driven Design.

¿Qué es DDD?

Domain-Driven Design

Enfoque de diseño que **prioriza el dominio y su lógica** frente a la tecnología.

- Nace para sistemas donde la **complejidad del negocio** es el problema principal.
- Alinea implementación con el modelo mental del negocio.
- Crea un **punto de comunicación** entre expertos de dominio y desarrolladores.

No es un framework ni un stack: es una forma de **modelar y acotar** software complejo.

Dos caras del DDD

Estratégico

- Ubiquitous Language
- Domain / Subdomain
- Bounded Context
- Context Mapping

¿Dónde van los límites? ¿Cómo se relacionan los modelos?

Táctico

- Entity / Value Object
- Aggregate
- Command / Domain Event
- (Repository, ...)

¿Cómo se estructura el modelo *dentro* de un contexto?

Mapa de la clase

1. **Ubiquitous Language** – un lenguaje compartido (con conflictos)
2. **Domain** → **Subdomain** → **BC** – Core / Supporting / Generic
3. **Context Mapping** – cómo se integran los contextos
4. **Puente a microservicios** – BC como frontera candidata
5. **Táctico mínimo** – Entity, VO, Aggregate, Command, Event
6. **EventStorming + taller** – del relato del negocio al modelo

Ubiquitous Language

Ubiquitous Language

Lenguaje **común y preciso** entre desarrolladores y expertos de dominio.

- Se usa en conversaciones, documentación **y código**.
- Se construye **en colaboración**; no lo inventan solo los devs.
- Refleja el dominio del problema, no la base de datos ni el framework.

Si en el código decís `OrderDTO` y el negocio dice “pedido de reposición”, el lenguaje **no es ubicuo**.

Hilo conductor: e-commerce de pedidos

A lo largo de la clase usamos un solo dominio:

Un cliente compra productos online.

El pedido atraviesa Ventas → Operaciones (inventario) → Facturación → Logística.

Ese flujo alcanza para practicar lenguaje, BCs, eventos y límites de servicio.

“Pedido”: ¿hablamos de lo mismo?

Área	Qué entiende por Pedido	Qué le importa
Ventas	Carrito confirmado por el cliente	ítems, cantidades, cliente
Operaciones	Solicitud de stock a reservar	disponibilidad, reserva
Facturación	Documento a cobrar	monto, medio de pago, estado de cobro
Logística	Unidad a despachar	domicilio, tracking, entrega

Misma palabra, **cuatro modelos**. Forzar uno solo corrompe las reglas de cada área.

Pregunta para la clase

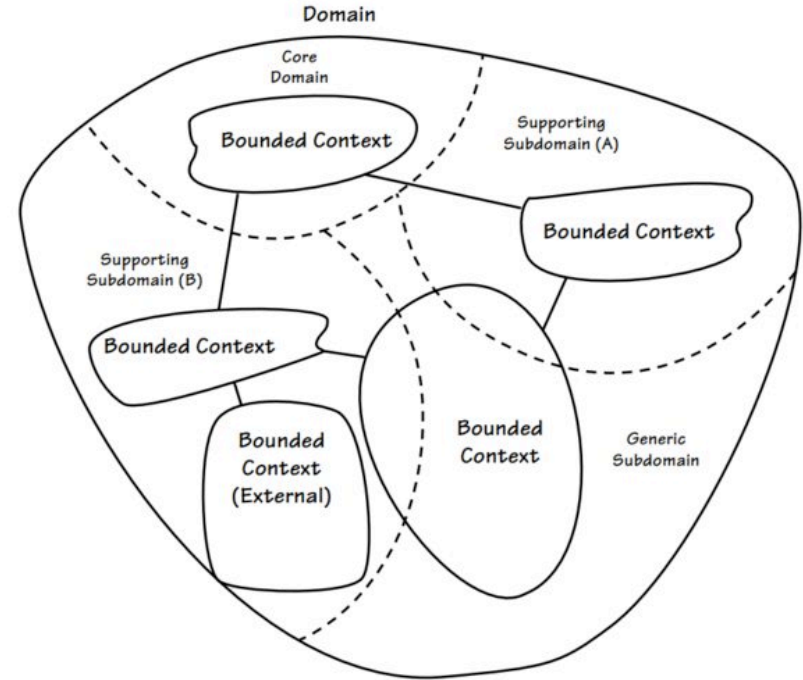
¿Cuántas definiciones de “Pedido” escuchaste en tu último proyecto?

- Si hay más de una y un solo servicio/modelo → señal de **límite mal puesto**.
- El Ubiquitous Language es **por Bounded Context**, no global a toda la empresa.

Domain, Subdomain y Bounded Context

Domain y Subdomains

- **Domain:** el área de conocimiento del negocio (aquí: comercio electrónico / fulfillment).
- No es código: es el **mundo de ideas y reglas** del problema.
- **Subdomains:** recortes del dominio general (ventas, inventario, cobros, envíos, ...).



Core / Supporting / Generic

No todos los subdominios merecen la misma inversión.

Tipo	Rol	En nuestro ejemplo
Core Domain	Diferencia competitiva; máxima atención al diseño	Pricing / reglas de venta (si es el DNA del negocio) o el flujo de fulfillment propio
Supporting	Necesario, específico del negocio, no diferenciador	Inventario interno, logística propia
Generic	Resuelto por el mercado; no reinventar	Pagos con tarjeta, email, autenticación

Microservicios “caros” se justifican más en **Core** y en Supporting crítico; en Generic preferí SaaS o módulos simples.

Bounded Context

Límite **explícito** dentro del cual un modelo de dominio es válido y consistente.

- Un mismo término puede modelarse distinto **fuera** del límite.
- Reduce conflictos de modelo y aclara reglas de negocio.
- Cada contexto puede (idealmente) corresponder a un equipo.
- La integración con otros contextos es **explícita** (APIs, eventos, traducción).

Domain vs Subdomain vs BC

- **Subdomain:** partición del **problema** de negocio.
- **Bounded Context:** partición de la **solución** (modelo + lenguaje + equipo).
- Idealmente se alinean 1:1, pero no siempre: un subdomain puede implementarse como varios BCs, o un BC puede abarcar un supporting + un generic adaptado.

El diagrama conceptual:

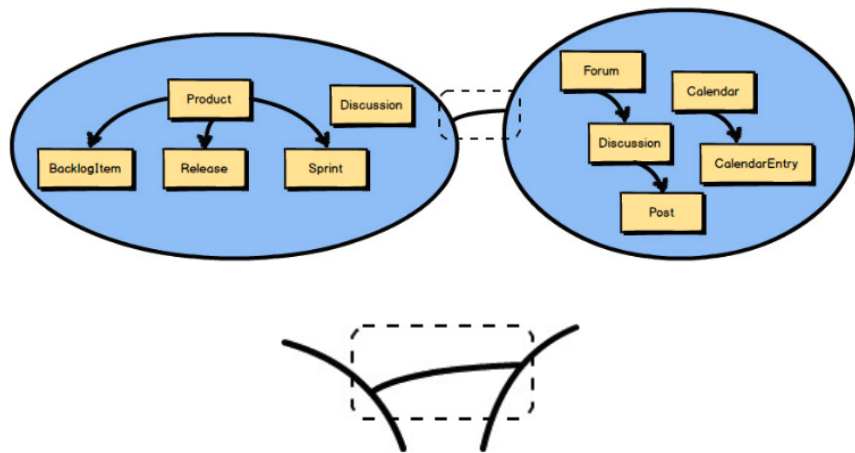
```
Domain
├── Subdomains (Core / Supporting / Generic)
│   ├── Bounded Contexts ← aquí vive el Ubiquitous Language
│   │   └── Modelo táctico (Aggregates, Events, ...)
```

Context Mapping

Context Mapping

Herramienta para visualizar **cómo interactúan** los Bounded Contexts.

- Marca límites de cada modelo.
- Muestra dependencias y dirección de influencia (upstream / downstream).
- Hace visibles conflictos y puntos de traducción.



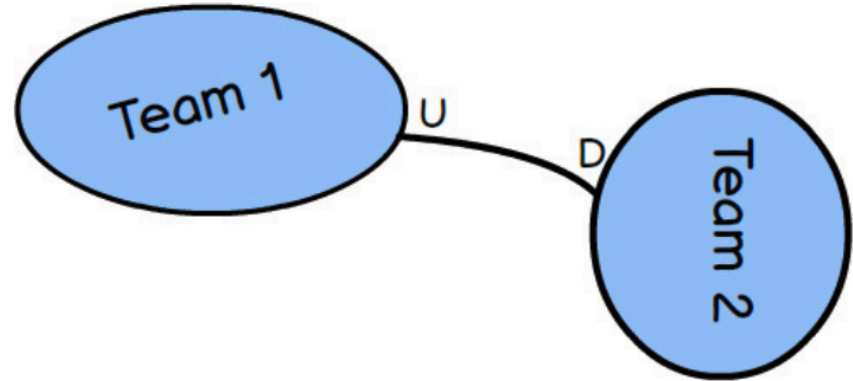
Customer / Supplier y Conformist

- **Customer / Supplier**

- El Supplier es **upstream** (provee capacidades).
- El Customer es **downstream** (depende de ellas).
- Negocian prioridades e interfaces.

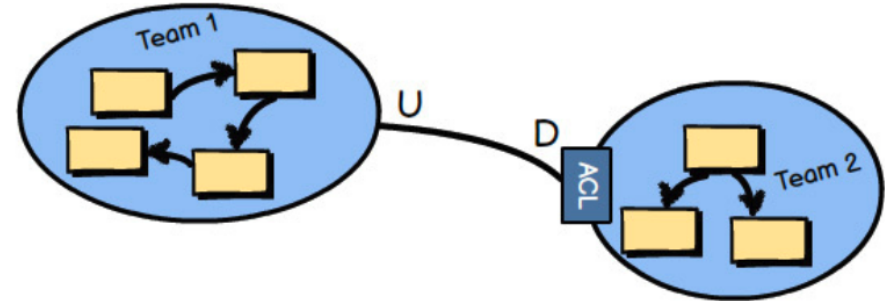
- **Conformist**

- El downstream **adopta** el modelo del upstream tal cual (sin traducción).
- Barato, pero acoplás el lenguaje ajeno.



Anticorruption Layer (ACL)

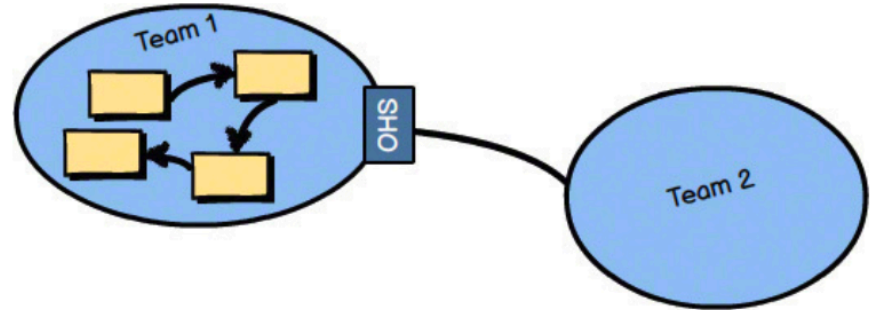
- Capa de **adaptación** entre dos modelos.
- Traduce el Ubiquitous Language del otro al nuestro.
- Protege el Core de modelos ajenos, legacy o de terceros.
- En microservicios: a menudo un adapter / gateway / anti-corruption service.



Open Host Service + Published Language

Open Host Service (OHS): un contexto expone capacidades a muchos consumidores.

- Protocolo e interfaz **bien definidos**.
- **Published Language:** el contrato público (JSON Schema, eventos, OpenAPI, AsyncAPI...), no el modelo interno crudo.
- El host publica un lenguaje de integración; cada consumidor puede traducir con ACL si hace falta.



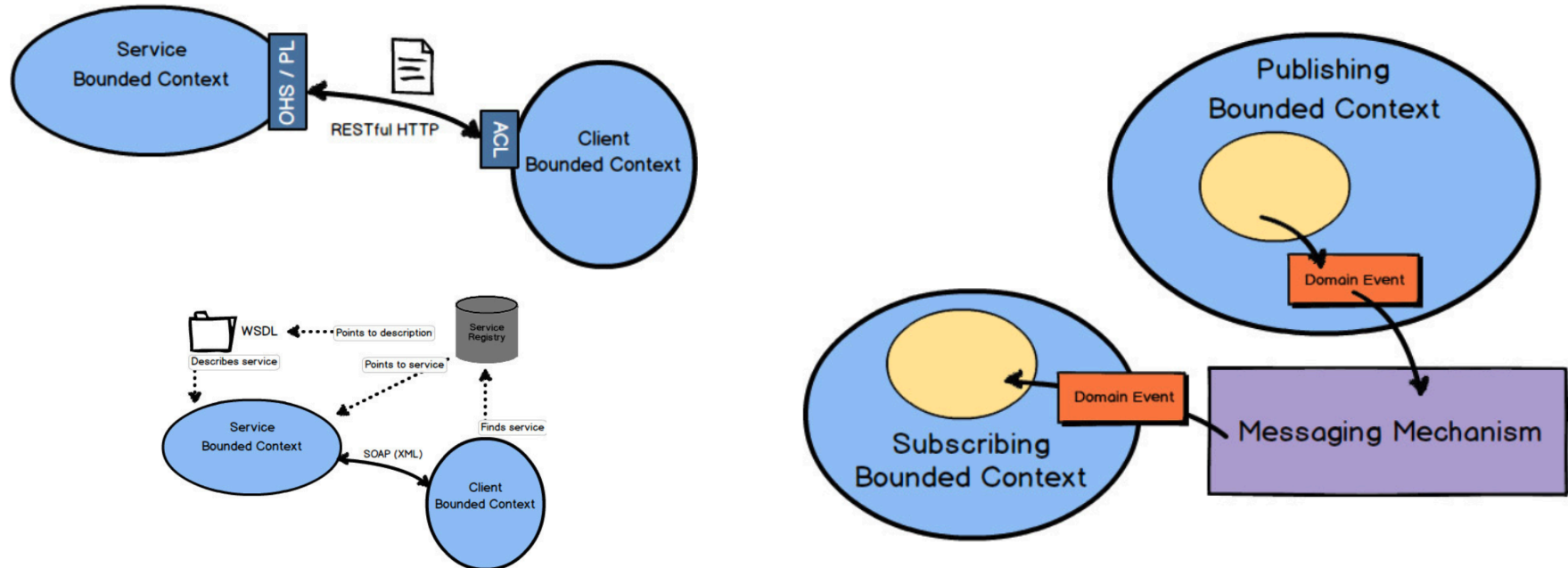
Otras relaciones

- **Partnership:** dos equipos con objetivos compartidos y sincronización fuerte.
- **Shared Kernel:** comparten un subconjunto de modelo; **ambos** lo evolucionan (costoso en coordinación).
- **Separate Ways:** deciden **no integrarse** aunque el negocio se solape; cada uno resuelve por su lado.

Separar caminos $\neq$ "compartimos la entidad": es aceptar duplicación o caminos paralelos a cambio de autonomía.



Integraciones frecuentes en la práctica



Síncrono (HTTP/gRPC) o asíncrono (Domain Events): el mapa de contextos guía **qué** integrás; la tecnología de la materia (Kafka, gateway) resuelve **cómo**.

Del BC al microservicio

Bounded Context $\neq$ microservicio

Bounded Context	Microservicio
Concepto de diseño/modelo	Concepto de runtime/despliegue
Define lenguaje y reglas	Define proceso, escala, fallos
Puede ser un módulo en un monólito	Puede mapear 1 BC $\rightarrow$ 1 servicio

Regla práctica: el BC es el **mejor candidato** a frontera de servicio.

No es obligatorio desplegar cada BC por separado el día 1 (monolito modular válido).

Señales de un buen límite

- Un equipo puede **cambiar el modelo interno** sin negociar con todos.
- El Ubiquitous Language es **estable dentro** y se traduce en el borde.
- Las transacciones fuertes caben **dentro** del aggregate/contexto.
- La integración exterior tolera latencia y fallos parciales (eventos, sagas).

Anti-patrones a evitar

- **Microservicios por tabla/entidad** (ya lo vimos al principio).
- **Shared Database** entre servicios → monolito distribuido.
- **Shared Kernel** demasiado grande → todos se sincronizan para cada change.
- **Un solo "PedidoService" global** con reglas de todas las áreas.
- Conformist al Core de otro sin ACL cuando el lenguaje ajeno te contamina.

DDD táctico (mínimo)

Entity

- Objeto con **identidad** que se mantiene en el tiempo.
- Dos instancias con el mismo id son “la misma cosa” aunque cambien atributos.
- Ejemplos en pedidos:
 - Cliente (id de cliente)
 - Pedido en Ventas (id de pedido de venta)
 - Envío en Logística (id de despacho)

La igualdad se define por **identidad**, no por todos los campos.

Value Object

- Se define por sus **atributos**; no tiene identidad propia.
- Inmutable en la práctica (se reemplaza, no se muta a medias).
- Ejemplos: Dinero (monto + moneda), Dirección, Cantidad, SKU.

```
Dinero(100, "ARS") = Dinero(100, "ARS")    ✓  
Pedido#42 = Pedido#42                      ✓ (por id)
```

Si dos "direcciones" idénticas son intercambiables → Value Object.

Si necesitás seguir "esta dirección particular a lo largo del tiempo" → reevaluar.

Aggregate

- **Cluster** de entidades y value objects tratados como una unidad.
- Tiene una **raíz (Aggregate Root)**; el exterior solo habla con ella.
- Frontera de **consistencia transaccional**: un comando modifica un aggregate a la vez.
- Ejemplo Operaciones: raíz ReservaInventario con líneas de ítems reservados.

```
[ReservaInventario] ← raíz
├─ LineaReserva (ítem, cantidad)
└─ Estado { pendiente | confirmada | liberada }
```

Command y Domain Event

Command		Domain Event
Naturaleza	Intención (imperativo)	Hecho ocurrido (pasado)
Ejemplo	AprobarPedido	InventarioReservado
Fallible	Puede rechazarse	Ya sucedió; se propaga
Alcance	Entra a un aggregate/contexto	Sale del contexto hacia otros

Flujo típico:

```
Command → Aggregate aplica reglas → Domain Event(s)
```

Los eventos son el puente natural hacia **comunicación asíncrona** y sagas (próximas clases).

Mínimo táctico en el flujo de pedidos

Contexto	Aggregate (ej.)	Eventos de dominio (ej.)
Ventas	PedidoVenta	PedidoCreado
Operaciones	ReservaInventario	InventarioReservado, InventarioInsuficiente, InventarioLiberado
Facturación	Cobro	PagoConfirmado, PagoRechazado, CreditoEmitido
Logística	Envio	EnvioProgramado, PedidoDespachado, EntregaConfirmada, EntregaFallida

Cada contexto nombra su mundo: no hay un único Pedido compartido.

8. EventStorming

¿Qué es EventStorming?

Técnica de modelado **rápida, colaborativa y visual** (Alberto Brandolini, ~2013).

- Junta negocio + desarrollo en el mismo muro (o pizarra digital).
- Parte de los **Domain Events** (pasado perfecto): *PedidoCreado*, *PagoConfirmado*.
- Produce alineación y documentación viva del flujo.
- Escala de “big picture” del proceso a diseño de aggregates y BCs.

Notación (Brandolini)

Actor / Persona

Command

Domain Event

Aggregate

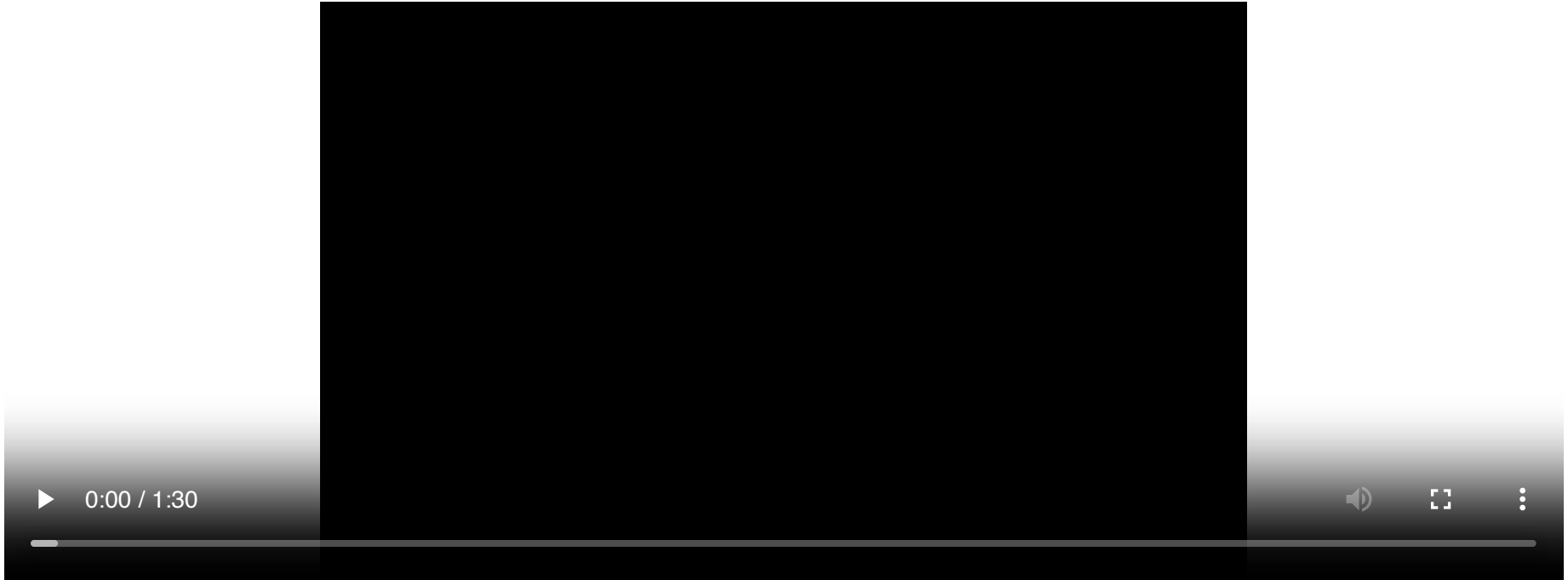
Policy
(whenever...
then...)

Sistema externo

Read Model /
Vista

Hotspot / Open
Issue

EventStorming en acción



90 minutos de sesión remota en ~90 segundos · @StolarczykT con Mariusz Gil, Andrzej Krzywda, Łukasz Szydło · Big Picture EventStorming (mrpicky.dev)

9. Taller guiado

Conversación acerca de un proceso

Se reúnen los gerentes de Ventas, Logística, Operaciones y Facturación para diseñar el sistema.



Requisitos – camino feliz

Cada uno cuenta lo que el sistema debe hacer, desde su punto de vista:

- **Gte. Ventas:** cuando el usuario presiona comprar, se genera un pedido con los ítems y la cantidad.
- **Gte. Operaciones:** al recibir un pedido, verifica disponibilidad, lo aprueba y **reserva** ítems en inventario.
- **Gte. Facturación:** los pedidos aprobados se facturan y se cobran por tarjeta (el cobro tarda); al confirmarse, el pedido queda **pagado** y pasa a Logística.
- **Gte. Logística:** los pedidos pagados se programan y despachan; al confirmar recepción, se informa a Ventas.

Requisitos – fallos

- **Operaciones:** sin stock → rechaza el pedido. Si Facturación rechaza → **libera** inventario. Si Logística devuelve → actualiza inventario.
- **Facturación:** cobro fallido → rechaza e informa a Operaciones. Envío fallido → **emite crédito** por lo pagado.
- **Logística:** no se puede entregar → devuelve ítems a Operaciones e informa a Facturación y Ventas.

Pregunta en clase: **¿quién “posee” el estado del pedido en cada momento?**

Paso 1 – Ubiquitous Language conflictivo

En el pizarrón, listas por área (sin unificar todavía):

Término	Ventas	Operaciones	Facturación	Logística
Pedido	carrito confirmado	solicitud de reserva	documento a cobrar	unidad a enviar
Aprobado	–	stock OK + reserva	listo para cobrar	–
Pagado	listo para promesa al cliente	–	cobro OK	habilitado a despachar
Rechazado	venta caída	sin stock / release	cobro fallido	–

No “arreglen” el lenguaje global: **acoten contextos**.

Paso 2 – Propuesta de Bounded Contexts

Bounded Context	Responsabilidad	Aggregate raíz (candidato)
Ventas / Compras	Capturar intención de compra	PedidoVenta
Inventario / Operaciones	Disponibilidad y reservas	ReservaInventario
Facturación	Cobro y créditos	Cobro
Logística	Planificación, despacho, entrega	Envio

Generic posible: **Pasarela de pagos** (sistema externo) detrás de ACL o OHS.

Paso 3 – Eventos del flujo

Camino feliz (pasado perfecto):

1. PedidoCreado
2. InventarioReservado
3. PagoConfirmado
4. EnvioProgramado → PedidoDespachado → EntregaConfirmada

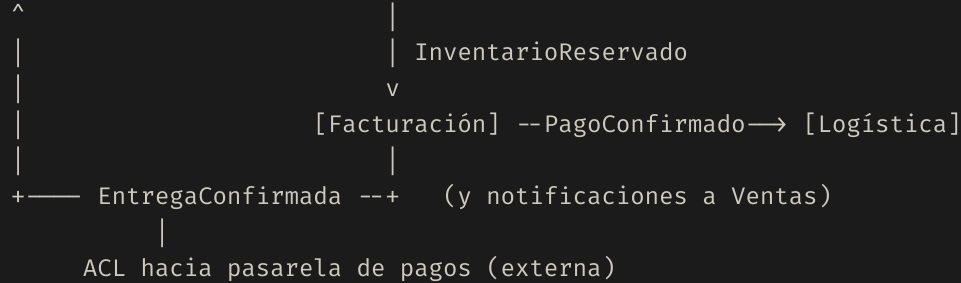
Fallos (policies “whenever... then...”):

- InventarioInsuficiente → informar a Ventas
- PagoRechazado → LiberarInventario (command) → InventarioLiberado
- EntregaFallida → liberar/actualizar inventario + CreditoEmitido

Estos eventos son candidatos a **mensajes entre servicios** en el módulo de comunicación.

Paso 4 – Mapa de contextos (borrador)

Ventas] --PedidoCreado--> [Inventario



Integraciones candidatas: **eventos asíncronos** entre BCs; **ACL** hacia el proveedor de cobros.

Mini-ejercicio (5-10 min)

En grupos:

1. Elijan **un** fallo del relato (ej. cobro fallido o entrega fallida).
2. Escriban 3-5 **Domain Events** y 1-2 **Commands**.
3. Digan qué BC **publica** y cuál **reacciona** (policy).
4. (Bonus) ¿Dónde pondrían un **hotspot** del EventStorming?

Después comparamos resultados en plenario.

10. Síntesis

Checklist de la clase

- ■ No corto servicios por tabla: busco **límites de modelo**.
- ■ Ubiquitous Language **por contexto**, con conflictos explícitos.
- ■ Clasifico subdominios: **Core / Supporting / Generic**.
- ■ Bounded Context = modelo + lenguaje + equipo (+ candidato a servicio).
- ■ Context map: ACL, OHS, Customer/Supplier, etc.
- ■ Aggregate = frontera de consistencia; Command in / Event out.
- ■ EventStorming para alinear negocio y diseño.

Lectura recomendada

Crash course visual y conciso sobre DDD (estrategia + táctica):

Complemento conceptual (Evans / Vernont / Brandolini) si querés profundizar en el TP de diseño.

Preview – lo que viene en la materia

Con BCs y Domain Events ya tenés el mapa conceptual:

Idea de hoy	Siguiente en el curso
Integración entre contextos	Patrones síncronos vs asíncronos, Saga
PagoConfirmado, InventarioLiberado	Kafka , topics, garantías de entrega
ACL / OHS	API Gateway, contratos, adapters
Límites de consistencia	Resiliencia, compensación, eventual consistency

Glosario express

- **Ubiquitous Language:** vocabulario compartido dev ↔ negocio (por BC).
- **Bounded Context:** límite de validez de un modelo.
- **Aggregate:** unidad de consistencia con raíz de acceso.
- **Domain Event:** hecho de negocio en pasado perfecto.
- **ACL:** traducción que protege el modelo propio.
- **Published Language:** contrato público de integración (≠ modelo interno).

Fin